



Generalized Additive Models

[Code](#) ▼

General Structure

Generalized Additive Models (GAM's) provide a general framework for adding of non-linear functions together instead of the typical linear structure of linear regression. GAM's can be used for either continuous or categorical target variables. The structure of GAM's are the following:

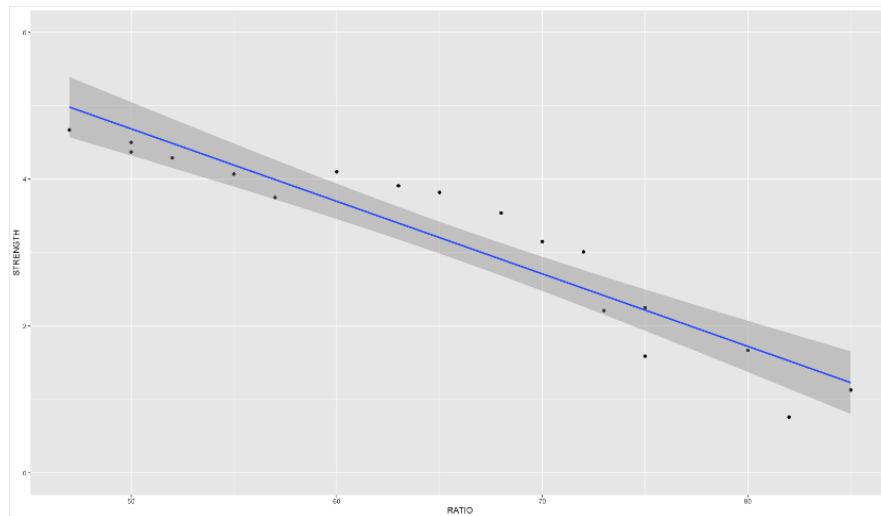
$$y = \beta_0 + f_1(x_1) + \dots + f_p(x_p) + \varepsilon$$

The $f_i(x_i)$ functions are complex, nonlinear functions on the predictor variables. GAM's add these complex, yet individual functions together. This allows for many complex relationships to try and model with to potentially predict your target variable better. We will examine a few different forms of GAM's below.

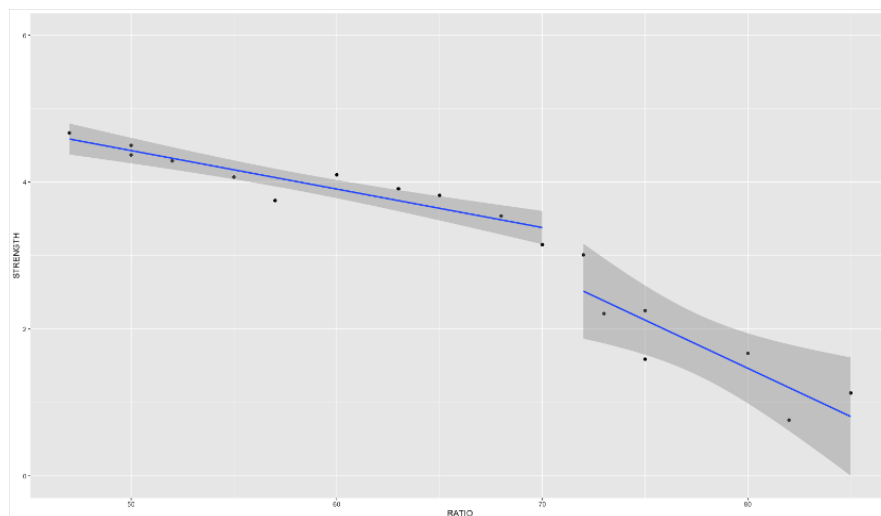
Piecewise Linear Regression

The slope of the linear relationship between a predictor variable and a target variable can change over different values of the predictor variable. The typical straight-line model $\hat{y} = \beta_0 + \beta_1 x_1$ will not be a good fit for this type of data.

Here is an example of a data set that exhibits this behavior - comprehensive strength of concrete and the proportion of water mixed with cement. The comprehensive strength decreases at a much faster rate for batches with a greater than 70% water/cement ratio.



Linear Regression



Piecewise Linear Regression

If you were to fit a linear regression as the first image above, it wouldn't represent the data very well. However, this is perfect for a **piecewise linear regression**. Piecewise linear regression is a model where there are different straight-line relationships for different intervals in the predictor variable. The following piecewise linear regression is for two slopes:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 (x_1 - k)x_2 + \varepsilon$$

The k value in the equation above is called the **knot value** for x_1 . The x_2 variable is defined as a value of 1 when $x_1 > k$ and a value of 0 when $x_1 \leq k$. With x_2 defined this way, when $x_1 \leq k$, the equation becomes $y = \beta_0 + \beta_1 x_1 + \varepsilon$. When $x_1 > k$, the equation gets a new intercept and slope: $y = (\beta_0 - k\beta_2) + (\beta_1 + \beta_2)x_1 + \varepsilon$.

Let's see this in each of our softwares!

R Python

Piecewise linear regression is built with typical linear regression functions with only some creation of new variables. In R, we can use the `lm` function on this cement data set. We are predicting **STRENGTH** using the **RATIO** variable as well the **X2STAR** variable which is $(x_1 - k)x_2$.

▼ Code

```
cement.lm <- lm(STRENGTH ~ RATIO + X2STAR, data = cement)
summary(cement.lm)
```

Call:

```
lm(formula = STRENGTH ~ RATIO + X2STAR, data = cement)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.72124	-0.09753	-0.00163	0.24297	0.49393

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	7.79198	0.67696	11.510	7.62e-09 ***
RATIO	-0.06633	0.01123	-5.904	2.89e-05 ***
X2STAR	-0.10119	0.02812	-3.598	0.00264 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

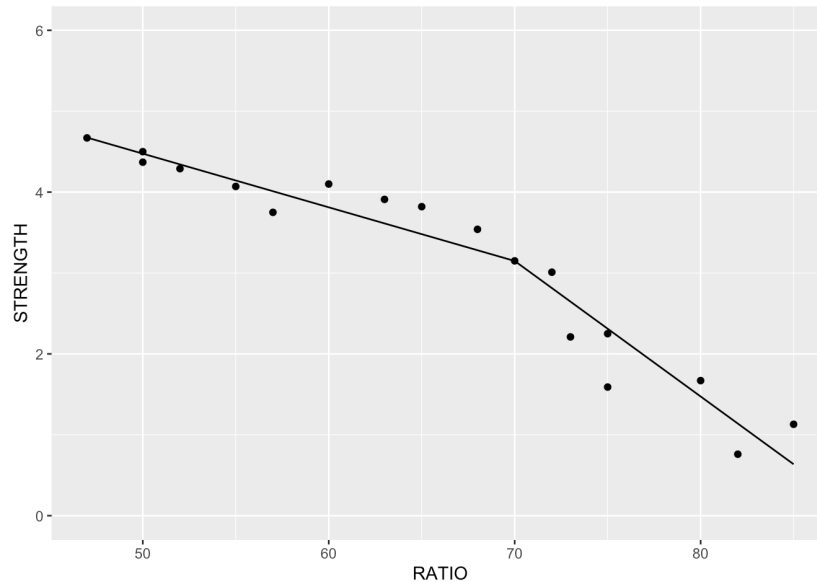
Residual standard error: 0.3286 on 15 degrees of freedom

Multiple R-squared: 0.9385, Adjusted R-squared: 0.9303

F-statistic: 114.4 on 2 and 15 DF, p-value: 8.257e-10

▼ Code

```
ggplot(cement, aes(x = RATIO, y = STRENGTH)) +
  geom_point() +
  geom_line(data = cement, aes(x = RATIO, y = STRENGTH)) +
  ylim(0,6)
```



We can see in the plot above how at the knot value of 70, the slope and intercept of the regression line changes.

The previous example dealt with piecewise functions that are continuous - the lines stay attached. However, you could make a small adjustment to the model to make the linear discontinuous:

$$y = \beta_0 + \beta_1 x_1 + \beta_2(x_1 - k)x_2 + \beta_3 x_2 + \varepsilon$$

With the addition of the same x_2 variable as previously defined on its own instead of attached to the $(x_1 - k)$ piece, the lines are no longer attached.

▼ Code

```
cement.lm <- lm(STRENGTH ~ RATIO + X2STAR + X2, dat
summary(cement.lm)
```

Call:

```
lm(formula = STRENGTH ~ RATIO + X2STAR + X2, data = cement)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.53167	-0.15513	0.06171	0.17239	0.49451

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	7.04975	0.68558	10.283	6.6e-08 ***
RATIO	-0.05240	0.01174	-4.463	0.000536 ***

```

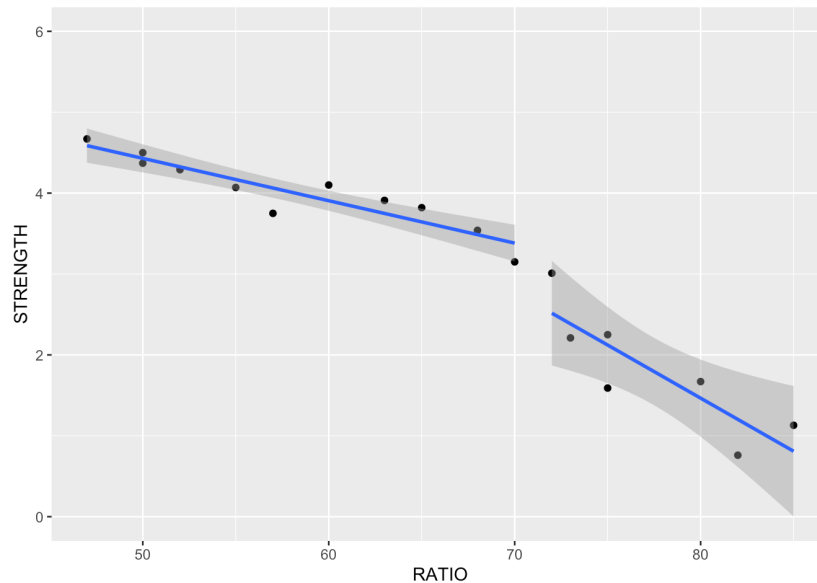
X2STAR      -0.07888    0.02686   -2.937  0.010830 *
X2          -0.60388    0.26877   -2.247  0.041302 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Residual standard error: 0.2916 on 14 degrees of freedom
 Multiple R-squared: 0.9548, Adjusted R-squared: 0.9451
 F-statistic: 98.57 on 3 and 14 DF, p-value: 1.188e-09

▼ Code

```
qplot(RATIO, STRENGTH, group = X2, geom = c('point'
```



The piecewise linear regression equation can be extended to have as many pieces as you want. An example with three lines (two knots) is as follows:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 (x_1 - k_1) x_2 + \beta_3 (x_1 - k_2) x_3 + \varepsilon$$

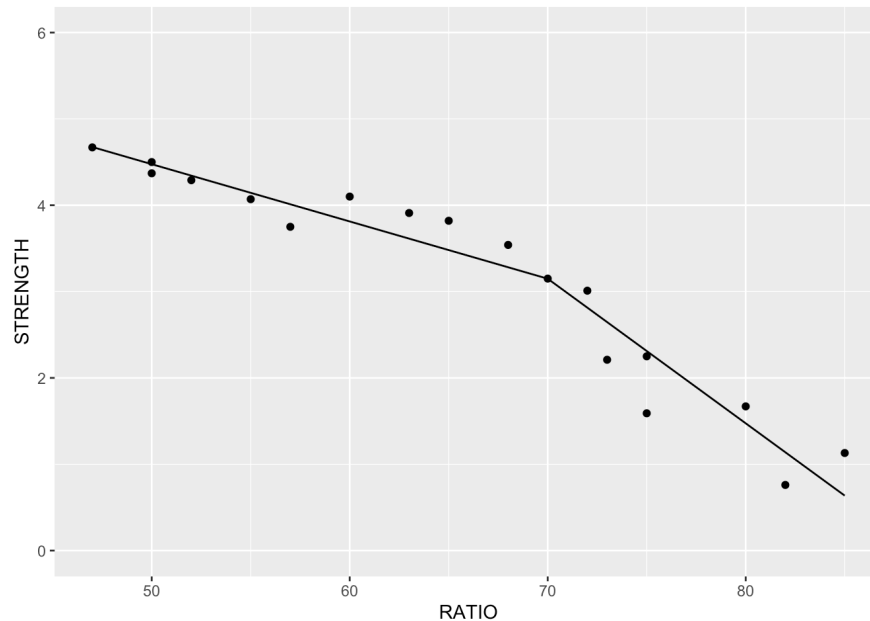
One of the problems with this structure is that we have to define the knot values ourselves. The next set of models can help do that for us!

MARS (and EARTH)

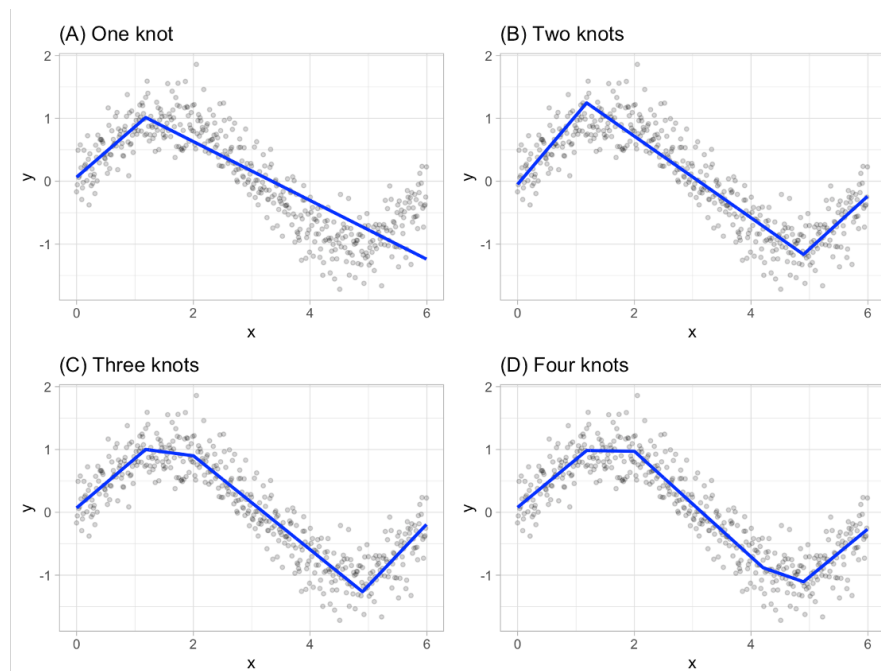
Multivariate adaptive regression splines (MARS) is a non-parametric technique that still has a linear form to the model (additive) but has nonlinearities and interaction between variables. Essentially, MARS uses piecewise regression approach to split into pieces then potentially uses either linear or nonlinear patterns for each piece.

MARS first looks for the point in the range of a predictor x_i where two

linear functions on either side of the point provides the least squared error (linear regression).



The algorithm continues on each piece of the piecewise function until many knots are found.



Example of Growing Number of Knots

This will eventually overfit your data. However, the algorithm then works backwards to "prune" (or remove) the knots that do not contribute significantly to out of sample accuracy. This out of sample accuracy calculation is performed by using the **generalized cross-validation** (GCV) procedure - a computational short-cut for leave-one-out cross-

validation. The algorithm does this for all of the variables in the data set and combines the outcomes together.

The actual MARS algorithm is trademarked by Salford Systems, so instead the common implementation in most softwares is **enhanced adaptive regression through hinges** - called EARTH.

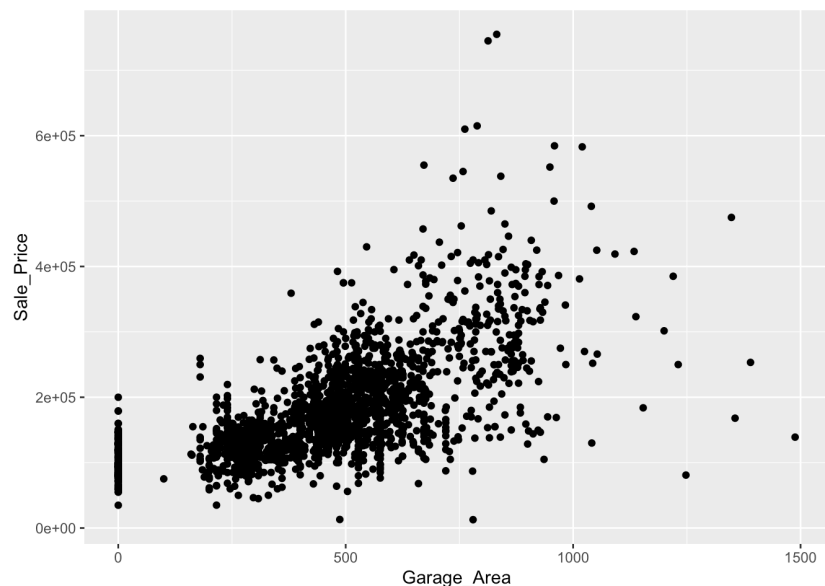
Let's see how to do this in each of our softwares!

R Python

Let's go back to our Ames housing data set and the variables we were working with in the previous section. One of the variables in our data set is `Garage_Area`. It doesn't have a straight-forward relationship with our target variable `Sale_Price` as seen by the plot below.

▼ Code

```
ggplot(training, aes(x = Garage_Area, y = Sale_Price))
  geom_point()
```



Let's fit the EARTH algorithm between `Garage_Area` and `Sale_Price`. In the `earth` package is the `earth` function. The input is similar to most modeling functions in R, a formula to relate predictor variables to a target variable and an option to define the data set being used. We will then look at a `summary` of the output.

▼ Code

```
library(earth)

mars1 <- earth(Sale_Price ~ Garage_Area, data = tra
summary(mars1)
```

```
Call: earth(formula=Sale_Price~Garage_Area, data=training)
```

	coefficients
(Intercept)	124159.039
h(286-Garage_Area)	-60.257
h(Garage_Area-286)	297.277
h(Garage_Area-521)	-483.642
h(Garage_Area-576)	733.859
h(Garage_Area-758)	-356.460
h(Garage_Area-1043)	-490.873

Selected 7 of 7 terms, and 1 of 1 predictors

Termination condition: RSq changed by less than 0.001 at 7 terms

Importance: Garage_Area

Number of terms at each degree of interaction: 1 6
(additive model)

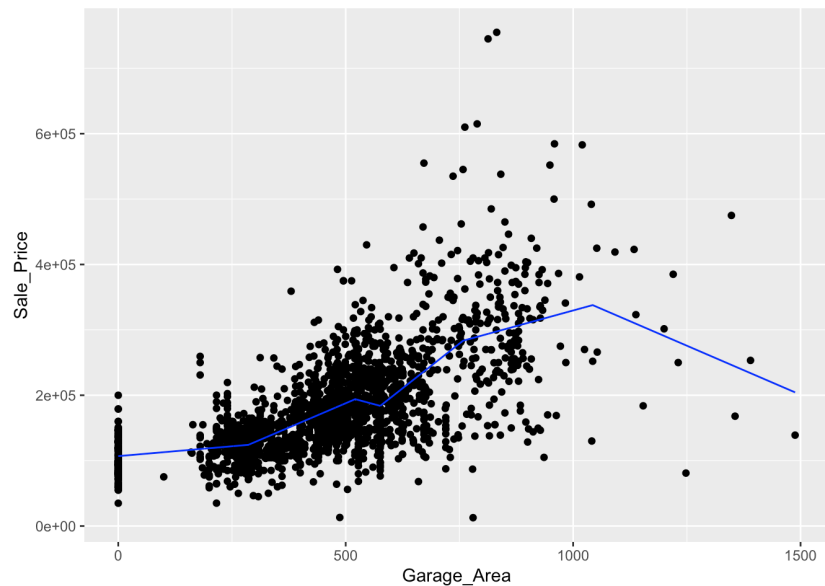
GCV 3427475346 RSS 6.94092e+12 GRSq 0.4492014 RSq 0.4556309

From the output above we see 6 pieces of the function defined by 5 knots. Those five knots correspond to the `Garage_Area - ____` values above. The coefficients attached to each of those pieces are the same as what we would have in piecewise linear regression. The bottom of the output also shows the generalized R^2 value as well as the typical R^2 value.

To visualize the piecewise relationship between `Garage_Area` and `Sale_Price` we can plot the predicted values on the scatterplot from above.

▼ Code

```
ggplot(training, aes(x = Garage_Area, y = Sale_Price)) +
  geom_point() +
  geom_line(data = training, aes(x = Garage_Area, y = Sale_Price))
```

We can see that the `Sale_Price` of the home stays relative steady for small values of `Garage_Area` but then increases to a point, before it begins to level off again.

Now let's build the algorithm on all the variables in the data set that we have. The `Sale_Price ~ .` notation tells the `earth` function to use all the variables in the data set to predict the `Sale_Price`.

▼ Code

```
mars2 <- earth(Sale_Price ~ ., data = training)
summary(mars2)
```

Call: `earth(formula=Sale_Price~., data=training)`

	coefficients
(Intercept)	319493.46
Central_AirY	20289.49
h(4-Bedroom_AbvGr)	9214.66
h(Bedroom_AbvGr-4)	-23009.05
h(Year_Built-1977)	1275.57
h(2004-Year_Built)	-336.64
h(Year_Built-2004)	5315.57
h(13869-Lot_Area)	-2.09
h(Lot_Area-13869)	0.22
h(First_Flr_SF-1600)	104.91
h(2402-First_Flr_SF)	-71.56
h(First_Flr_SF-2402)	-176.61
h(1523-Second_Flr_SF)	-53.13
h(Second_Flr_SF-1523)	426.63
h(Half_Bath-1)	-45378.31
h(2-Fireplaces)	-14408.56
h(Fireplaces-2)	-26072.58

```
h(Garage_Area-539)          101.97
h(Garage_Area-1043)        -294.30
h(Gr_Liv_Area-2049)         65.21
h(Gr_Liv_Area-3194)        -159.79
```

Selected 21 of 24 terms, and 10 of 14 predictors

Termination condition: Reached nk 29

Importance: First_Flr_SF, Second_Flr_SF, Year_Built,
Garage_Area, ...

Number of terms at each degree of interaction: 1 20
(additive model)

```
GCV 1033819964    RSS 2.036439e+12    GRSq 0.8338641    RSq
0.8402842
```

Now that all of the variables have been added in, we see a lot of them remaining in the model to predict `Sale_Price`. There are knot values defined for all of the variables that are in the model. Right below the knot values in the output above we see that only 10 of the 14 original variables were used in the final model. Two lines below that we see variables listed by importance. We will look at this more below. Not surprisingly, the R^2 and generalized R^2 has increased with the addition of all these new variables. Notice how `Garage_Area` has different knot values then when we ran the algorithm on `Garage_Area` alone. That is because the algorithm prunes the knots with all of the variables in the model. Apparently, some of the other variables being in the model means we don't need as many knots in the `Garage_Area` variable.

Let's talk more about that variable importance metric in the above output. For each model size (1 term, 2 terms, etc.) there is one "subset" model - the best model for that size. EARTH ranks variables by how many of these "best models" of each size that variable appears in. The more subsets (or "best models") that a variable appears in, the more important the variable. We can get this full output using the `evimp` function on our `earth` model object.

▼ Code

```
evimp(mars2)
```

	nsubsets	gcv	rss
First_Flr_SF	20	100.0	100.0
Second_Flr_SF	19	71.7	71.9
Year_Built	18	50.9	51.3
Garage_Area	17	34.3	35.0
Fireplaces	16	31.0	31.7
Gr_Liv_Area	15	27.6	28.4
Central_AirY	12	20.0	20.9
Bedroom_AbvGr	11	18.1	19.0
Lot_Area	10	16.2	17.2
Half_Bath	4	7.4	8.2

The **nsubsets** above is the number of subsets that the variable appears in. The **rss** above stands for residual sum of squares (or sum of squares error) is a scaled version of the decrease in residual sum of squares relative to the previous subset. Since it is scaled, the top variable always has a value of 100 while the remaining ones decrease from there. The **gcv** value is an approximation of **rss** on leave-one-out cross-validation and is also scaled.

Interpretability of relationships between predictor variables and the target variable starts to get more complicated with the EARTH (or MARS) algorithm. You can plot the relationship as we see above, but those relationships can still be rather complicated and hard to explain to a client.

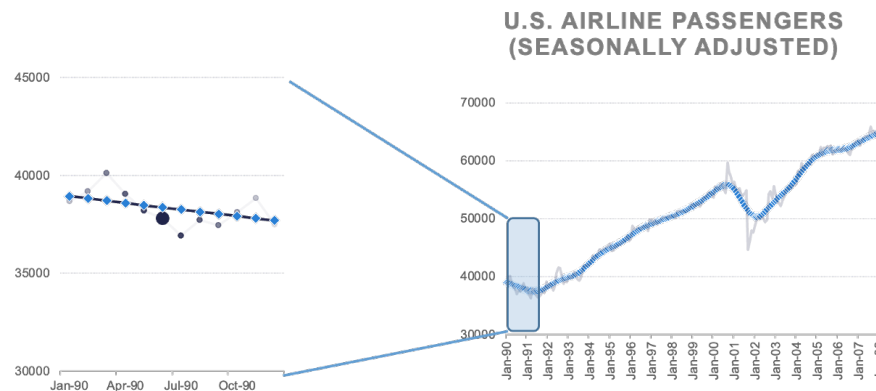
Smoothing

Generalized additive models can be made up of any non-parametric function of the predictor variables. Another popular technique is to use **smoothing functions** so the piecewise linear regressions are not so jagged. The following are different types of smoothing functions:

- LOESS (localized regression)
- Smoothing splines & regression splines

LOESS

Locally estimated scatterplot smoothing (LOESS) is a popular smoothing technique. The idea of LOESS is to perform weighted linear regression in small windows of a scatterplot of data between two variables. This weighted linear regression is done around each point as the window moves from the low end of the scatterplot values to the high end. An example is shown below:



LOESS Regression

The predictions of these regression lines in each window are connected together to form the smoothed curve through the scatterplot as shown above.

Smoothing Splines

Smoothing splines take a different approach as compared to LOESS. Smoothing splines have a knot at every single observation for piecewise regression which leads to overfitting. There is a penalty parameter used to counterbalance the “wobble” of the spline curve.

Smoothing splines try to find the function $s(x_i)$ that optimally fits x_i to the target variable through the following equation:

$$\min \sum_{i=1}^n (y_i - s(x_i))^2 + \lambda \int s''(t_i)^2 dt$$

By thinking of $s(x_i)$ as a prediction of y , the front half of the equation is equal to the sum of squared errors in your model. The second half of the equation above has the λ penalty applied to the integral of the second derivative of the smoothing function. To conceptually think of this second derivative we can think of it as the “slope of slopes” which is large when the curve has a lot of “wobble”. The optimal value of the λ penalty is estimated with another approximation of leave-one-out cross-validation.

Regression splines are just a computationally nicer version of smoothing splines so they will not be covered in detail here.

Let’s see how to do GAM’s with splines in each of our softwares!

R Python

Continuing to use our Ames housing data set, we will build a `gam` using the `mgcv` package in R. Similar to previous functions the inputs are the formula for the model and the `data =` option to define the data set. We will also use the `summary` function to view the output. Inside of the formula, we use the `s` function to inform the `gam` function to which variables should have splines fit to them.

▼ Code

```
library(mgcv)

gam1 <- mgcv::gam(Sale_Price ~ s(Garage_Area), data =
summary(gam1)
```

Family: gaussian
Link function: identity

```

Formula:
Sale_Price ~ s(Garage_Area)

Parametric coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  180897      1290    140.2   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
              edf Ref.df    F p-value
s(Garage_Area) 8.134  8.769 192.5  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
' 1

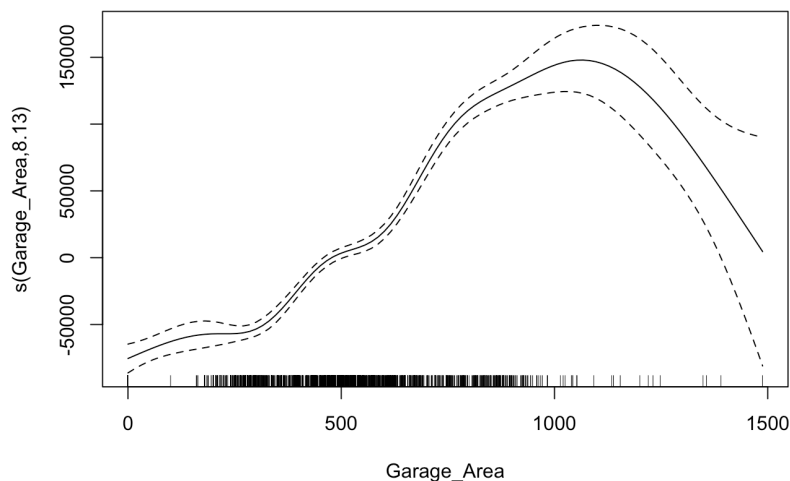
R-sq.(adj) =  0.451   Deviance explained = 45.3%
GCV = 3.4301e+09   Scale est. = 3.4148e+09   n = 2051

```

From the output above we see two different sections - a section for coefficients that are not involved in splines and a section for smoothing terms. The p-value attached to the spline of `Garage_Area` shows the significance of that variable to the model as a whole. Similar to the EARTH algorithm, we can view a plot of the relationship between the variable and its predictions of the target. Here we use the `plot` function on the `gam` model object.

▼ Code

```
plot(gam1)
```



This nonlinear and complex relationship between `Garage_Area` and `Sale_Price` is similar to the plot we saw earlier with EARTH. This shouldn't be too surprising. Both algorithms are trying to relate these two variables together, just in different ways.

Let's build out a GAM with all of the variables in our data set. The categorical variables are entered as either character variables or with the `factor` function. The continuous variables are defined with the spline function `s`.

▼ Code

```
gam2 <- mgcv::gam(Sale_Price ~ s(Bedroom_AbvGr, k =
                             s(Year_Built) +
                             s(Mo_Sold) +
                             s(Lot_Area) +
                             s(First_Flr_SF) +
                             s(Second_Flr_SF) +
                             s(Garage_Area) +
                             s(Gr_Liv_Area) +
                             s(TotRms_AbvGrd) +
                             Street +
                             Central_Air +
                             factor(Fireplaces) +
                             factor(Full_Bath) +
                             factor(Half_Bath)
                             , method = 'REML', data = trainin
summary(gam2)
```

Family: gaussian
Link function: identity

Formula:

```
Sale_Price ~ s(Bedroom_AbvGr, k = 5) + s(Year_Built) +
s(Mo_Sold) +
  s(Lot_Area) + s(First_Flr_SF) + s(Second_Flr_SF) +
s(Garage_Area) +
  s(Gr_Liv_Area) + s(TotRms_AbvGrd) + Street +
Central_Air +
  factor(Fireplaces) + factor(Full_Bath) +
factor(Half_Bath)
```

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	136140	19681	6.917	6.19e-12

StreetPave	27689	12710	2.178	0.0295 *
Central_AirY	18012	3168	5.685	1.50e-08

factor(Fireplaces)1	14070	1666	8.443	< 2e-16

```

factor(Fireplaces)2    27137      3146    8.626 < 2e-16
***
factor(Fireplaces)3    15705     10552    1.488  0.1368
factor(Fireplaces)4   -79595     31469   -2.529  0.0115 *
factor(Full_Bath)1     -5342     14528   -0.368  0.7131
factor(Full_Bath)2    -11075     14827   -0.747  0.4552
factor(Full_Bath)3      1225     15786    0.078  0.9381
factor(Full_Bath)4    -16272     24326   -0.669  0.5036
factor(Half_Bath)1       2102       2206    0.953  0.3408
factor(Half_Bath)2    -38507       9111  -4.226 2.48e-05
***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
              edf Ref.df      F p-value
s(Bedroom_AbvGr) 2.653  3.165 18.789 <2e-16 ***
s(Year_Built)     6.445  7.543 101.759 <2e-16 ***
s(Mo_Sold)         1.516  1.868   0.993  0.4507
s(Lot_Area)        7.186  8.193  11.726 <2e-16 ***
s(First_Flr_SF)   8.063  8.765  15.548 <2e-16 ***
s(Second_Flr_SF)  8.212  8.818   7.806 <2e-16 ***
s(Garage_Area)     7.426  8.328  21.654 <2e-16 ***
s(Gr_Liv_Area)     8.545  8.882  14.834 <2e-16 ***
s(TotRms_AbvGrd)  3.805  4.738   1.921  0.0783 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.851  Deviance explained = 85.6%
-REML = 23957  Scale est. = 9.2392e+08  n = 2051

```

The top half of the output has the variables not in splines, while the bottom half has the spline variables.

There are some variables with high p-values that could be removed from the model. One of the benefits of the `gam` function from the `mgcv` package is the `select` option. If we set `select = TRUE` then the model will penalize variables' edf values. You can think of an edf value almost like a polynomial term. The selection technique will zero out this edf value - essentially, zeroing out the variable itself.

An example is shown below where the `Mo_Sold` variable is essentially zeroed from the model.

▼ Code

```

sel.gam2 <- mgcv::gam(Sale_Price ~ s(Bedroom_AbvGr,
                                     s(Year_Built) +
                                     s(Mo_Sold) +
                                     s(Lot_Area) +

```

```

s(First_Flr_SF)
s(Second_Flr_SF)
s(Garage_Area) +
s(Gr_Liv_Area) +
s(TotRms_AbvGrd)
Street +
Central_Air +
factor(Fireplace
factor(Full_Bath
factor(Half_Bath
, method = 'REML', select = T
summary(sel.gam2)

```

Family: gaussian
Link function: identity

Formula:

```

Sale_Price ~ s(Bedroom_AbvGr, k = 5) + s(Year_Built) +
s(Mo_Sold) +
s(Lot_Area) + s(First_Flr_SF) + s(Second_Flr_SF) +
s(Garage_Area) +
s(Gr_Liv_Area) + s(TotRms_AbvGrd) + Street +
Central_Air +
factor(Fireplaces) + factor(Full_Bath) +
factor(Half_Bath)

```

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	138290	19777	6.993	3.67e-12

StreetPave	27999	12938	2.164	0.0306 *
Central_AirY	17771	3209	5.538	3.46e-08

factor(Fireplaces)1	13513	1694	7.974	2.55e-15

factor(Fireplaces)2	26758	3216	8.319	< 2e-16

factor(Fireplaces)3	15390	10733	1.434	0.1518
factor(Fireplaces)4	-73384	31848	-2.304	0.0213 *
factor(Full_Bath)1	-7674	14559	-0.527	0.5982
factor(Full_Bath)2	-13216	14816	-0.892	0.3725
factor(Full_Bath)3	3140	15766	0.199	0.8422
factor(Full_Bath)4	-36178	23069	-1.568	0.1170
factor(Half_Bath)1	2612	2222	1.176	0.2399
factor(Half_Bath)2	-37561	9267	-4.053	5.25e-05

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

Approximate significance of smooth terms:

edf	Ref.df	F	p-value
-----	--------	---	---------


```

s(Bedroom_AbvGr) 2.4004      4 14.886 < 2e-16 ***
s(Year_Built)    6.9457      9 83.935 < 2e-16 ***
s(Mo_Sold)        0.0380      9  0.004  0.34177
s(Lot_Area)       7.5606      9 11.776 < 2e-16 ***
s(First_Flr_SF)  8.6024      9 32.355 < 2e-16 ***
s(Second_Flr_SF) 0.9441      9  1.706 4.15e-06 ***
s(Garage_Area)    6.6895      9 19.622 < 2e-16 ***
s(Gr_Liv_Area)    4.8394      9  7.623 < 2e-16 ***
s(TotRms_AbvGrd) 3.7644      9  1.297  0.00801 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.845   Deviance explained = 84.9%
-REML = 24081   Scale est. = 9.6643e+08   n = 2051

```

Summary

In summary, GAM's are good models to use for prediction, but explanation becomes more difficult and complex. Some of the advantages of using GAM's:

- Allows nonlinear relationships without trying out many transformations manually
- Improved predictions
- Limited "interpretation" still available
- Computationally fast for small numbers of variables

There are some disadvantages though:

- Interactions are possible, but computationally intensive
- Not good for large number of variables so prescreening is needed
- Multicollinearity still a problem.